

OpenOCD 源码解读

Joe

November 13, 2025

目录

1	OpenOCD 源码框架	1
1.1	Configure 文件的解析部分	1
1.2	OpenOCD 与 GDB 的通信	2
1.3	OpenOCD 访问 Debug Module 寄存器	4
1.4	JTAG 物理信号的解析	7
1.5	OpenOCD 源代码调试环境的在 Ubuntu 中的搭建	8
1.6	小结	10
2	OpenOCD 重要结构	10
2.1	数据结构	10
2.2	函数	13
3	OpenOCD Flash 操作	13
3.1	Flash 配置命令	13
3.2	Flash Handler 函数注册	15
3.3	FLASH 刷写准备	15
3.4	Telnet 刷写 flash	16
3.4.1	PROBE 命令:	16
3.4.2	WRITE-FLASH 命令	17
3.4.3	校验数据	19
3.5	小结	19

表格目录

1 DTM 寄存器 4

Listings

1 OpenOCD 源码框架

OpenOCD 的代码结构整体分成两大部分:

1. 对于输入的 `configure` 文件的解析. 该文件用来配置 JTAG 通信中各个参数, 甚至包括对目标板的一些初始化操作也在文件中预先定义好, 比如对于 FLASH 以及 RAM 的一些初始化操作等. 该文件的语法是 TCL 的一个子集, OpenOCD 集成了一个 C 语言编写的 TCL 的解释器 (Jim). OpenOCD 在启动后会解析该 TCL 语法的 `configure` 文件, 然后调用相应的预先注册好的 `command handler` 函数执行特定的命令 (下文会涉及 `handler` 函数的注册).
2. OpenOCD 是通过 `socket` 的方式和 GDB 进行通信的. 在初始化阶段也包含了 `socket` 的建立. 初始化完成后, 就会进入 `server_loop()` 函数中, 在这个函数的大循环中, 接收来自 GDB 的命令 (GDB 命令的解析下文会涉及). 对每个命令字符串, 调用相应的函数进行特定的处理. 比如设置断点, 单步运行等等. 通过一层层的 `call stack`, 最终将命令发送给 MCU 的 Debug Module 模块中. 当然在处理完成后会通过 `socket` 将 `response` 的消息反馈给 GDB.

1.1 Configure 文件的解析部分

OpenOCD 的初始化部分的 `call stack` 如图1所示. `command_registrants[]` 表中存放的是需要进行注册的所有 `command handler` 函数. 当 `configure` 文件在解析处理的过程中会最终调用这些 `handler` 函数进行处理.

```
## the main entry
openocd_main()
  setup_command_handler()
  command_init()
  (*command_registrants[i])(cmd_ctx)
  ## "startup.tcl.inc" initialization
  ## "register all commands below in the table"
  static const command_registrant_t command_registrants[] = {
    &openocd_register_commands,
    &server_register_commands,
    &gdb_register_commands,
    &log_register_commands,
    &transport_register_commands,
    &interface_register_commands,
    &target_register_commands,
    &flash_register_commands,
    &hand_register_commands,
    &pld_register_commands,
    &mflash_register_commands,
    &cti_register_commands,
    &dap_register_commands,
    NULL
  };
  util_init()
  openocd_thread()
  server_loop()
  flash_free_all_banks()
  gdb_service_free()
  server_free()
  unregister_all_commands()
```

图 1: OpenOCD 初始化

我们以注册 `trace` 的 `handler` 为例, 图2是 `trace handler` 的结构. 注意其中的 `name` 项与 `handler` 项是对应的, Jim 模块在查找特定 `handler` 的时候就是通过 `name` 来定位.

- 如果 `mode = COMMAND_EXEC` 则表明该 `handler` 函数是在命令窗中通过输入命令才会触发的预注册函数.
- 如果 `mode = COMMAND_CONFIG` 则表明该 `handler` 函数是在 OpenOCD 在启动阶段解析 `cfg` 文件的时候才会触发的预注册函数.
- 如果 `mode = COMMAND_ANY` 则表明以上两种情况下都会触发的预注册 `handler` 函数.

```

#####
## the trace command list below
#####
static const struct command_registration trace_exec_command_handlers[] = {
{
    .name = "history",
    .handler = handle_trace_history_command,
    .mode = COMMAND_EXEC,
    .help = "display trace history, clear history or set size",
    .usage = "['clear'|size]",
},
{
    .name = "point",
    .handler = handle_trace_point_command,
    .mode = COMMAND_EXEC,
    .help = "display trace points, clear list of trace points, "
        "or add new tracepoint at address",
    .usage = "['clear'|address]",
},
},
COMMAND_REGISTRATION_DONE
};
static const struct command_registration trace_command_handlers[] = {
{
    .name = "trace",
    .mode = COMMAND_EXEC,
    .help = "trace command group",
    .usage = "",
    .chain = trace_exec_command_handlers,
},
},
COMMAND_REGISTRATION_DONE
};

```

图 2: OpenOCD Trace Handler

图3是 command handler 经过注册后的拓扑结构图, 其中 command_context 是一个全局量. 如图3所示, 所有的 handler 是分组链接在一起的. Jim 在查找 handler 的时候, 比如查找 mflash 中的 init, 先找到 mflash, 然后再找到 init 的.



图 3: OpenOCD Handler 链

1.2 OpenOCD 与 GDB 的通信

函数 server_loop() 调用结构如图4所示, 本身是一个大循环, 接收来自 GDB 或 Telnet 等通过 socket 传过来的数据. 调用 server->input() 对接收到的数据进行解析, 然后再调用特定的函数进行处理.

```

server_loop()
{
    service_input(c)
    gdb_input(c)
    gdb_input_inner()
    gdb_get_packet()
    gdb_thread_packet()
    gdb_get_registers_packet()
    gdb_set_registers_packet()
}

```

图 4: OpenOCD GDB 通信

这里可以举几个例子来分析 GDB 的命令是如何得到执行的:

- 图5是设置软件断点的处理逻辑: 其中"Z0,100310,4"是来自 GDB 发送过来的命令字符串, 其中"Z0"表示设置软断点,"0"表示软断点(若为"1"则表示硬断点),"100310"为16进制值,表示断点设置的地址,"4"表示该地址处的机器码长度为4个字节."\$OK#9a"表示 OpenOCD 处理完该命令后要反馈给 GDB 的消息. 本文章是基于 RISC-V 的平台, 通过如下图的调用栈可以看到, 对

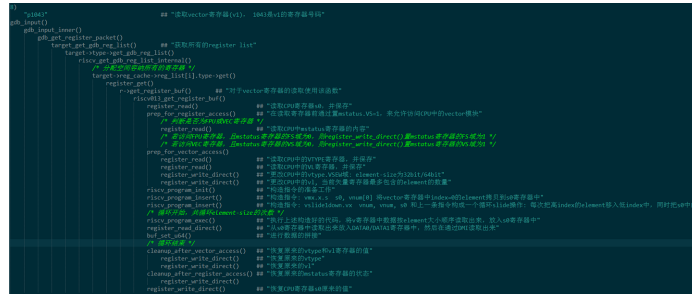


图 8: OpenOCD GDB 读寄存器

1.3 OpenOCD 访问 Debug Module 寄存器

这一部分涉及的内容是与处理器平台相关的, 本文主要基于 RISC-V 平台, 但是原理都是相通的. 一般来讲所有的 debug 逻辑的处理都是基于对 MCU 内部寄存器的访问来实现的.

1. OpenOCD 对于 RISC-V 内部的 CSR 等寄存器的访问需要借助 DTM 寄存器来进行访问的. 当需要访问 JTAG 寄存器的时候, 需要使用如表 1 中 Debug Module Interface Access 寄存器 (addr=0x11) 来进行访问.

表 1: DTM 寄存器

Name	Address
BYPASS	0x00
IDCODE	0x01
DTM Control&Status(dtmcs)	0x10
Debug Module Interface Access(dmi)	0x11
Reserved(BYPASS)	0x12
Reserved(BYPASS)	0x13
Reserved(BYPASS)	0x14
Reserved(BYPASS)	0x15
Reserved(BYPASS)	0x16
Reserved(BYPASS)	0x17
BYPASS	0x1F

2. OpenOCD 对于 DMI 寄存器的写访问通过如图 9 所示命令格式.

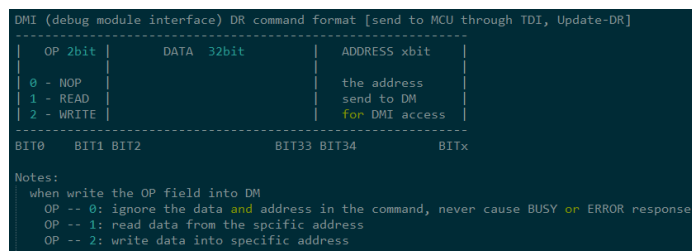


图 9: OpenOCD DMI 写格式

3. 对于 DMI 寄存器的读访问通过如图 10 所示命令格式, 比如要访问 RISC-V 的 mstatus 寄存器, 首先需要按照下面 4,5,6 项所描述的那样构造 abstract command, 该 command 中包含 mstatus 寄存器的 index 值. 将 abstract command 寄存器的地址 0x17 以及刚构造好的 abstract command 按照上图显示的 DR command format 重新构造. 然后就可以通过 transport_write() 函数将数据发送到 MCU 的 Debug Module 中了.

DMI (debug module interface) DR command format [recv from MCU through TDO, Capture-DR]

STATE 2bit	DATA 32bit	ADDRESS 7bit
0 - success	the returned data from	
2 - failed	DM as result of the	
3 - busy	previous operation	

BIT0 BIT1 BIT2 BIT33 BIT34 BITx

when read the OP field from DM
 OP -- 0: the previous operation is successfully.
 OP -- 1: reserved
 OP -- 2: previous operation failed, data scanned into DMI in this access, will be ingored
 OP -- 3: an operation comes while DMI request is in progress.

图 10: OpenOCD DMI 读格式

4. 要访问 Bus Registers 中任一一个寄存器, 必须通过上面讲述的 Debug Module Interface Access 来进行间接的访问. 其 Bus Register 列表如图 11 所示.

detail name	name	address
Abstract Data 0	(data0)	0x04
Abstract Data 11	(data11)	0x0f
Debug Module Control	(dmcontrol)	0x10
Debug Module Status	(dmstatus)	0x11
Hart Info	(hartinfo)	0x12
Halt Summary 1	(haltsum1)	0x13
Hart Array Window Select	(hawindowse1)	0x14
Hart Array Window	(hawindow)	0x15
Abstract Control and Status	(abstractcs)	0x16
Abstract Command	(command)	0x17
Abstract Command Autoexec	(abstractauto)	0x18
Configuration String Pointer 0	(confstrptr0)	0x19
Configuration String Pointer 1	(confstrptr1)	0x1a
Configuration String Pointer 2	(confstrptr2)	0x1b
Configuration String Pointer 3	(confstrptr3)	0x1c
Next Debug Module	(nextdm)	0x1d
Program Buffer 0	(progbuf0)	0x20
Program Buffer 15	(progbuf15)	0x2f
Authentication Data	(authdata)	0x30
Halt Summary 2	(haltsum2)	0x34
Halt Summary 3	(haltsum3)	0x35
System Bus Address 127:96	(sbaddress3)	0x37
System Bus Access Control and Status (sbcs)	(sbcs)	0x38
System Bus Address 31:0	(sbaddress0)	0x39
System Bus Address 63:32	(sbaddress1)	0x3a
System Bus Address 95:64	(sbaddress2)	0x3b
System Bus Data 31:0	(sbdata0)	0x3c
System Bus Data 63:32	(sbdata1)	0x3d
System Bus Data 95:64	(sbdata2)	0x3e
System Bus Data 127:96	(sbdata3)	0x3f
Halt Summary 0	(haltsum0)	0x40

图 11: OpenOCD Bus 寄存器

5. 其中”Abstract Command”寄存器在访问其他 MCU 内部寄存器时比较关键. 其结构如图 12 所示



图 12: OpenOCD Abstract CMD

6. 当需要访问 MCU 内部的特定寄存器的时候, 比如需要访问 mstatus(RISC-V 内部的状态寄存器) 时, 需要首先构造 abstract command 字段, 该字段 32 位, 其中包含了要访问的寄存器的 index 的值. OpenOCD 发送给 Debug Module 后, 会触发硬件将该寄存器值存放到”Abstract Data 0”寄存

器中. OpenOCD 只需要直接读取即可. 如图13所示是借助 Bus Register 读取特定寄存器的函数调用关系.

```

"utilize the abstract to read the specific register"
register_read_abstract()
|
|
V
"check the register number supported or not"
|
|
V
access_register_command() "construct the command to access the register"
  "set cmdtype area"
  "set aarsize area"
  "set regno area, the specific register index is here"
  "set transfer area"
|
|
V
execute_abstract_command()
|
|
+----+
|
V
dmi_write_exec() "send command to 'abstract command register' (address = 0x17)"
|
|
V
wait_for_idle() "read 'abstractcs' register to check busy or not (address = 0x16)"
  "when command is written, the busy-bit will be set, cleared until command is finished"
  "loop here for a short while to wait the busy state is cleared!!!"
|
|
V
"check the 'cmderr' in the 'abstractcs' register"
"to make sure it has error or not"
"if 'cmderr != 0', error happen, write cmderr area to clear it"
|
|
+----+
|
V
read_abstract_arg() "here we will read the value of the specific register"
  dmi_read() "read the specific register value from DATA0 and/or DATA1 (for 64-bit)"
|
|
V
"END"

```

图 13: OpenOCD Bus 寄存器读调用关系

7. 图14是写寄存器的调用栈关系.

```

register_write_abstract()
|
|
V
"check the specific register supported or not"
|
|
V
access_register_command() "construct the command to access the register"
  "set cmdtype area"
  "set aarsize area"
  "set regno area, the specific register index is here"
  "set transfer area"
  "set write area"
|
|
V
write_abstract_arg() "write the value into DATA0/DATA1"
  dmi_write()
|
|
V
execute_abstract_command() "same as upwards flow-chart"
  "transfer and write area is setted in abstract command"
  "it will cause: copy data from DATA0 into specific register"
|
|
V
"END"

```

图 14: OpenOCD Bus 寄存器写调用关系

8. 当正常访问 MCU 内部寄存器的逻辑处理失败后,OpenOCD 会启动异常处理流程来达到目的.

限于篇幅的原因就不再贴调用流程图, 简单描述如下:

- 首先 OpenOCD 需要构造一条汇编指令, 该指令的作用是读取寄存器的内容到一个特定的 MCU 寄存器中, 比如 csrr s0,mstatus. 该条指令的含义是将 RISC-V 的 mstatus 寄存器内容读取到 s0 寄存器中, 它是一个机器码, 该机器码包含了源操作数和目的操作数.

- 将该机器码发送到 Debug Module 的 bus 寄存器 progbufn(program buffer [n=0~15]) 中。
- 构造 abstract command 字段, 将该字段发送到”Abstract Comand” 寄存器中, 触发存放在 program buffer 中的机器码被 MCU core 获取, 从而得到执行, 数据存放到了 s0 寄存器中。
- 构造 abstract command 字段, 将该字段发送到”Abstract Comand” 寄存器中, 触发将 s0 寄存器中的数据拷贝到”Abstract Data 0” 寄存器中。
- OpenOCD 直接读取”Abstract Data 0” 寄存器中的数据, 从而获取最终需要的寄存器中的数据。

1.4 JTAG 物理信号的解析

上述所有对寄存器的访问都是通过 JTAG 的物理引脚来传递信号的。一般的 JTAG 物理信号主要包含四个 pin 脚:

- TCK: 移位时钟引脚
- TMS: TAP (test access point) 的状态机切换信号引脚
- TDI: 数据移位输入
- TDO: 数据移位输出

在 OpenOCD 与 Debug Module 两侧各维护着一个状态一致的状态机, 如图 15所示, 数据的真正输入输出是在”SHIFT-DR” 和”SHIFT-IR” 状态完成的。

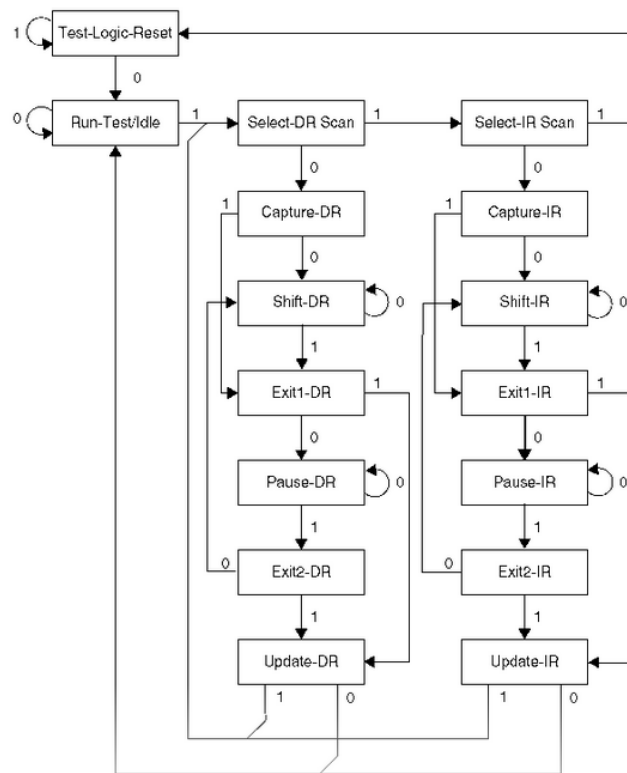


图 15: Jtag 状态机

图16是一个传输的例子: 发送 0x01 地址给 Debug Module.

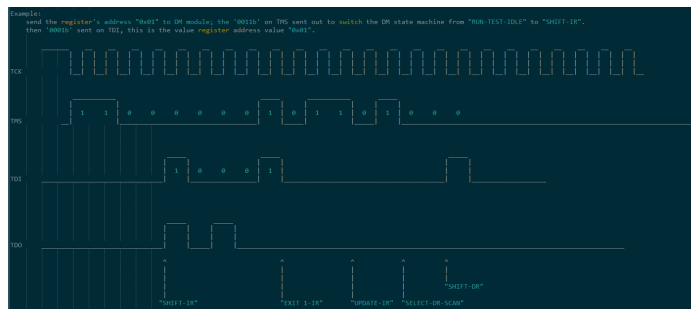


图 16: Jtag 传输 DM 命令

1.5 OpenOCD 源代码调试环境的在 Ubuntu 中的搭建

- 要编译 OpenOCD 源码需要使用如下命令安装软件包

```
sudo apt-get install gawk gperf grep gettext\
python python-dev automake bison flex \
texinfo help2man libtool libtool-bin
```

- 使用如下命令下载 OpenOCD 源代码

```
git clone https://github.com/riscv/riscv-openocd.git
```

- 使用如下命令安装 ddd 调试器

```
sudo apt-get install ddd
```

- 按照 OpenOCD 源代码文件夹下的步骤进行编译, 从而生成需要的 elf 文件, 位置在 xx/riscv-openocd/src/openocd
- 打开一个终端, 启动 gdbserver 运行 OpenOCD, 命令行如下, 其中 openocd 就是存放在 src 文件下刚刚编译出来的 elf 文件, 此时终端中会显示监听的端口号, 该端口号需要待会使用. 这个监听号是随机的, 每次都不一样, 这点需要注意

```
gdbserver:0 ./openocd -f openocd.cfg
```

- 另起一个终端, 启动 ddd, 命令行如下, 此时会跳出 ddd 的调试窗口如图 17 所示, 其中 target remote :39097 就是刚才的监听端口号.

```
ddd --debugger gdb
```

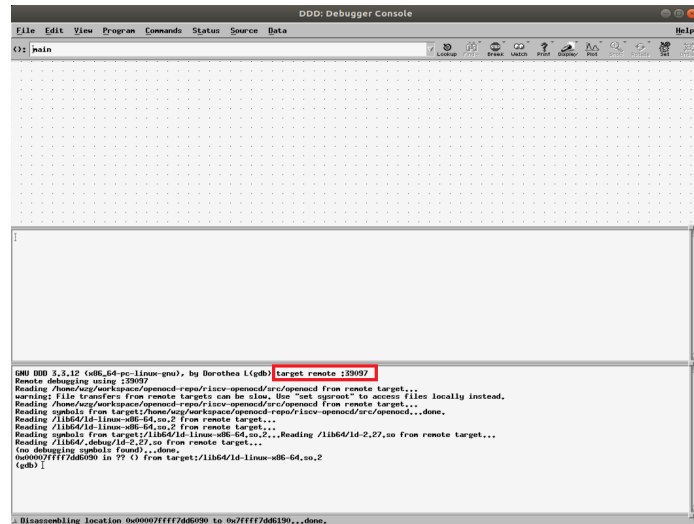


图 17: OpenOCD 软件调试

- 在 ddd 的这个调试窗口中可以输入 gdb 的各种调试命令
 - b <line number> 在代码中的某一行设置断点
 - si 单步调试一行汇编代码
 - s 单步调试一行 C 代码
 - n 继续走下一行
 - i b 查看设置了多少个断点
 - d <断点号> 删除特定的断点
 - c continue 继续运行
- 在 ddd 中调试源代码如图 18 所示, 右键点击某个变量, 选中下拉菜单中的 display 项, 就可以在变量窗口显示某个变量的值. 这比在 gdb 的黑漆漆的窗口中使用 p 命令强多了. 而且 ddd 可以将结构体变量中成员的链接关系以图形化的方式显示出来, 非常的直观, 这是笔者喜欢该调试工具一个地方. 让寄存器窗口弹出来, 选择工具栏中: Status->Registers; 查看函数调用栈的方式, 选择工具栏: Status->Backtrace

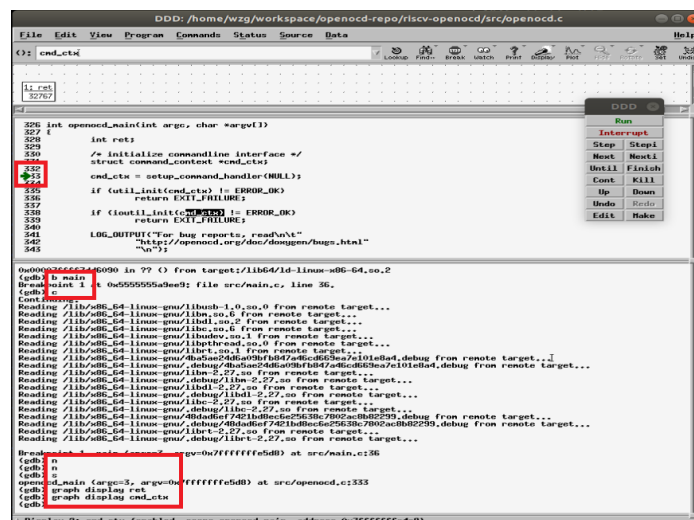


图 18: OpenOCD ddd 调试窗

- 上述的步骤只是将 OpenOCD 启动起来了, 在 ddd 的命令行窗口中输入 continue 将其启动起来, 就可以看到 OpenOCD 的 listening 端口已经显示出来了. 此时需要启动 risc-v 对应的 gdb, 也就是 risc-v 的 tool-chain 中的 gdb 工具. 启动方式很简单, 直接在新打开的命令行窗口中输入特定的命令, 如: riscv32-xxx-elf-gdb 即可. 然后使用 target remote :3333 就可以建立好 GDB 和 OpenOCD 的 socket 通信了. 通过 riscv-xxx-elf-gdb 调试不停的发送特定的调试命令, 在 ddd 中就可以调试 OpenOCD 中收到该命令时的处理动作了.

1.6 小结

对于 OpenOCD 源码的分析到这里就基本结束了. 其实最关键的部分还是需要对于特定处理器的 Debug Module 中各个控制逻辑的熟悉程度. 比如设置一个断点到 CPU 中, 需要设置哪些 DM 中的寄存器以及按照什么样的逻辑顺序来操作. 这些都需要了然于胸, 才能写出健壮的, 适配于特定处理器的 OpenOCD.

但大部分情况下, 我们只需要了解 OpenOCD 的基本工作原理即可. 在了解的基础上, 当 JTAG 通信工作不正常时可以定位到问题的原因就可以了. 在这里扩展的谈谈 GDB 与 OpenOCD 以及 Debug-Module 之间的互动关系: 在 OpenOCD 建立起了 GDB 与 DebugModule 之间的通信链路以后, 在调试的时候, 需要使用 GDB 将所要调试的 ELF 文件进行加载, GDB 的加载过程其实分两个主要部分, 一个是将 ELF 文件中的符号表信息进行提取, 这些信息会保留在 GDB 内部, GDB 后续进行单步调试就是依赖与这些信息的. 另一个是将去掉了符号表等信息剩下的 binary 通过已经建立好的链路下载到 CPU 侧. 在开始进行调试的时候, 由于 GDB 内部已经得到了所有符号的信息, 代码从哪个地址开始执行, 每行代码所在的地址这些都是知道的. 比如单步运行, GDB 只需要发送单步运行的命令即可, 单步运行前的起始位置 GDB 是知道的, 单步运行只不过是运行到下一个位置而已; 还比如设置断点到某行代码处, 该行代码所处的地址 GDB 根据所保留的符号信息是很清楚的, 只需要将要设置的断点处的地址发送给 OpenOCD 即可, OpenOCD 就会将后续的事情处理完毕; OpenOCD 其实在整个过程中除了建立链路外, 还起到了一个转译命令的作用, 将通信链路两端发送来的数据通过转译再传递出去.

2 OpenOCD 重要结构

2.1 数据结构

- struct command_context

```
struct command_context {
    Jim_Interp *interp;
    enum command_mode mode;
    struct target *current_target;
    /* The target set by 'targets xx' command or the latest created */
    struct target *current_target_override;
    /* If set overrides current_target
     * It happens during processing of
     * 1) a target prefixed command
     * 2) an event handler
     * Pay attention to reentrancy when setting override.
     */
    command_output_handler_t output_handler;
    void *output_handler_priv;
    struct list_head *help_list;
};
```

- enum command_mode

```
enum command_mode {  
    COMMAND_EXEC,  
    COMMAND_CONFIG,  
    COMMAND_ANY,  
    /* error condition */  
    COMMAND_UNKNOWN = -1,  
};
```

- struct target

```
struct target {  
    struct target_type *type; /* target type definition (name, access functions) */  
    char *cmd_name;           /* tcl Name of target */  
    struct jtag_tap *tap;     /* where on the jtag chain is this */  
    int32_t coreid;          /* which device on the TAP? */  
    char *current_target_name;  
  
    /** Should we defer examine to later */  
    bool defer_examine;  
  
    /**  
     * Indicates whether this target has been examined.  
     *  
     * Do @b not access this field directly, use target_was_examined()  
     * or target_set_examined().  
     */  
    bool examined;  
  
    /**  
     * true if the target is currently running a downloaded  
     * "algorithm" instead of arbitrary user code. OpenOCD code  
     * invoking algorithms is trusted to maintain correctness of  
     * any cached state (e.g. for flash status), which arbitrary  
     * code will have no reason to know about.  
     */  
    bool running_alg;  
  
    struct target_event_action *event_action;  
  
    bool reset_halt; /* attempt resetting the CPU into the halted mode? */  
    target_addr_t working_area; /* working area (initialised RAM). Evaluated  
                                * upon first allocation from virtual/physical address. */  
    bool working_area_virt_spec; /* virtual address specified? */  
    target_addr_t working_area_virt; /* virtual address */  
    bool working_area_phys_spec; /* physical address specified? */  
    target_addr_t working_area_phys; /* physical address */  
    uint32_t working_area_size;      /* size in bytes */  
};
```

```

bool backup_working_area; /* whether the content of the working area has to be preserved */
struct working_area *working_areas; /* list of allocated working areas */
enum target_debug_reason debug_reason; /* reason why the target entered debug state */
enum target_endianness endianness; /* target endianness */
/* also see: target_state_name() */
enum target_state state; /* the current backend-state (running, halted, ...) */
struct reg_cache *reg_cache; /* the first register cache of the target (core regs) */
struct breakpoint *breakpoints; /* list of breakpoints */
struct watchpoint *watchpoints; /* list of watchpoints */
struct trace *trace_info; /* generic trace information */
struct debug_msg_receiver *dbgmsg; /* list of debug message receivers */
bool dbg_msg_enabled; /* debug message status */
void *arch_info; /* architecture specific information */
void *private_config; /* pointer to target specific config data (for jim_configure hook) */
struct target *next; /* next target in list */
bool verbose_halt_msg; /* display async info in telnet session. Do not display
                        * lots of halted/resumed info when stepping in debugger. */
bool halt_issued; /* did we transition to halted state? */
int64_t halt_issued_time; /* Note time when halt was issued */

/* ARM v7/v8 targets with ADIV5 interface */
bool dbgbase_set; /* By default the debug base is not set */
uint32_t dbgbase; /* Really a Cortex-A specific option, but there is no
                  * system in place to support target specific options
                  * currently. */
bool has_dap; /* set to true if target has ADIV5 support */
bool dap_configured; /* set to true if ADIV5 DAP is configured */
bool tap_configured; /* set to true if JTAG tap has been configured
                    * through -chain-position */

struct rtos *rtos; /* Instance of Real Time Operating System support */
bool rtos_auto_detect; /* A flag that indicates that the RTOS has been specified as "auto"
                      * and must be detected when symbols are offered */
/* Track when next to poll(). If polling is failing, we don't want to
 * poll too quickly because we'll just overwhelm the user with error
 * messages. */
struct backoff_timer backoff;
unsigned int smp; /* Unique non-zero number for each SMP group */
struct list_head *smp_targets; /* list all targets in this smp group/cluster
                              * The head of the list is shared between the
                              * cluster, thus here there is a pointer */
bool smp_halt_event_postponed; /* Some SMP implementations (currently Cortex-M) stores
                              * 'halted' events and emits them after all targets of
                              * the SMP group has been polled */

/* the gdb service is there in case of smp, we have only one gdb server
 * for all smp target
 * the target attached to the gdb is changing dynamically by changing

```

```

    * gdb_service->target pointer */
    struct gdb_service *gdb_service;
    /* file-I/O information for host to do syscall */
    struct gdb_fileio_info *fileio_info;
    char *gdb_port_override; /* target-specific override for gdb_port */
    int gdb_max_connections; /* max number of simultaneous gdb connections */

    /* The semihosting information, extracted from the target. */
    struct semihosting *semihosting;
    bool resethalt_during_init;
    unsigned int etrace_trigger_index;
};

```

- struct command

```

struct command {
    char *name;
    command_handler_t handler;
    Jim_CmdProc *jim_handler;
    void *jim_handler_data;
    /* Command handlers can use it for any handler specific data */
    struct target *jim_override_target;
    /* Used only for target of target-prefixed cmd */
    enum command_mode mode;
};

```

- command_handler_t

```

typedef __COMMAND_HANDLER((*command_handler_t));
#define __COMMAND_HANDLER(name, extra ...) \
    int name(struct command_invocation *cmd, ## extra)

```

2.2 函数

3 OpenOCD Flash 操作

3.1 Flash 配置命令

openocd 的源码库在“<https://gitee.com/riscv-mcu/riscv-openocd.git>”。我们一般在使能 OpenOCD 的刷写功能的时候, 需要在 OpenOCD 的 cfg 文件中添加如下两条语句:

```

$_TARGETNAME configure -work-area-phys 0xaaaaaaaa -work-area-size 0xbbbbbbbb -work-area-backup 0
flash bank spi_flash fspi 0xcccccccc 0 0 0 $_TARGETNAME 0xdddddddd

```

这两条语句会在配置参数的同时, 注册一系列的 handler 函数. 对于第一条语句的解析: 图19是 OpenOCD 中的代码调用栈

- -work-area-phys 字符串后的数字会被解析出来, 放入 target->working_area_phys 变量中, 该变量存放的是 Core 上一个物理起始地址,
- -work-area-size 后面跟的是未来要以这个物理起始地址开始分配的一个空间的大小;

- -work-area-backup 决定了是否要对 Core 上那个空间的数据进行备份 (但感觉没有大的用处, OpenOCD 一般都会做备份动作).

```

jim_target_configure() "command-name='configure'"
target_configure()
jim_getopt_nvp() "从nvp_config_opts[]表中获取到 '-work-area-phys'/'-work-area-size'/'-work-area-backup' 项后"

V
TCFG_WORK_AREA_PHYS
+----> target_free_all_working_areas()
jim_getopt_wide() "获取跟在后面的数字 0x80000000"
"配置好参数: target->working_area_phys=0x80000000 以及 target->working_area_spec=true"
jim_NewIntObj() "构造一个实例"
jim_SetResult() "将构造的实例插入解析的表中 jim_getopt_info()->interp"

V
TCFG_WORK_AREA_SIZE
+----> target_free_all_working_areas() "释放所有前面分配的旧资源"
jim_getopt_wide() "获取跟在后面的数字 0x80000000"
"配置好参数: target->working_area_size=0x80000000"
jim_NewIntObj() "构造一个实例"
jim_SetResult() "将构造的实例插入解析的表中 jim_getopt_info()->interp"

+----> target_free_all_working_areas() "释放所有前面分配的旧资源"
jim_getopt_wide() "获取跟在后面的数字 0"
"配置好参数: target->backup_working_area = (!w)"
jim_NewIntObj() "构造一个实例"
jim_SetResult() "将构造的实例插入解析的表中 jim_getopt_info()->interp"

V
...

```

图 19: Flash 配置参数解析

对于第二条语句的解析: OpenOCD 在启动的时候注册了一系列的函数 handler, 主要分两大类: 一类是为了解析 config 文件中的语句而注册的 handler 函数, 另一类是为了在 OpenOCD 运行阶段解析输入进来的命令而注册的 handler 函数. 可以看到由于第二条语句中包含 "flash bank" 字符串, 因此通过遍历图20显示的定義关系, 预先注册的 handler 函数 handle_flash_bank_command() 就会被调用到. 用它来解析第二条语句后续的字段.

```

static const struct command_registration flash_config_command_handlers[] = {
{
.name = "bank",
.handler = handle_flash_bank_command,
.mode = COMMAND_CONFIG,
.usage = "bank_id driver_name base_address size_bytes "
"chip_width_bytes bus_width_bytes target "
"[driver_options ...]",
.help = "Define a new bank with the given name, "
"using the specified NOR flash driver.",
},
{
.name = "init",
.mode = COMMAND_CONFIG,
.handler = handle_flash_init_command,
.help = "Initialize flash devices.",
.usage = "",
},
{
.name = "banks",
.mode = COMMAND_ANY,
.handler = handle_flash_banks_command,
.help = "Display table with information about flash banks.",
.usage = "",
},
{
.name = "list",
.mode = COMMAND_ANY,
.jim_handler = jim_flash_list,
.help = "Returns a list of details about the flash banks.",
},
},
COMMAND_REGISTRATION_DONE
};

static const struct command_registration flash_command_handlers[] = {
{
.name = "flash",
.mode = COMMAND_ANY,
.help = "NOR flash command group",
.chain = flash_config_command_handlers,
.usage = "",
},
},
COMMAND_REGISTRATION_DONE
};

```

图 20: Flash 命令注册

图21是函数 handle_flash_bank_command() 的调用关系, 这里 CMD_ARGV[] 中保存的就是第二条语句中的后续字段. 依靠字符串 "fespi" 从全局表 flash_drivers[] 中找到对应的 flash driver, 这个表中

包含各种的 `flash-driver`, 是根据名称进行查找的, 其中的“`&fespi_flash`”项就是工作在 RISC-V 平台上的项. 找到该项后, 通过 `register_commands()` 函数将其中的函数进行注册. 在函数的最后, 将新建立的 `flash_bank{}` 实例加入到全局变量 `flash_banks` 链表中, 它的 `name` 就是我们在命令里面定义好的“`spi_flash`”, 后续要搜索这个链表时, 就是通过 `name` 来查找的.

```
#OPENAMD_HANDLER(handle_flash_bank_command)
// 在驱动中openpci的flash bank是硬编码的
get_target(CMD_ARGV[5]) // "获取target": 是取它的 TARGETNAME 的名字来确认target的。
flash_driver_find_by_name() // "根据名字 ARGV[0]=!-feapi": 从flash_drivers列表中查找与对应的flash driver。我们是 feapi_flash
// get_flash_bank_by_name() 函数调用，它返回一个指向 flash bank 的指针。
register_command() // 注册命令，即 feapi_flash 中的 flash_drivers 进程。
CALL_NAME("feapi", c-target, c-driver, c-base, c-size, c-chipwidth, c-bus width 等参数进行赋值。
// 添加新的 addded_value 或 c-varied value 给 c-command 列表。
c-callname_HANDLER(driver+flash_bank_command) // 添加新命令名称 feapi_flash_bank_command()
// feapi_flash_bank 函数。并赋值; 我们 feapi_flash_driver = feapi_info
// CMD_HANDLE_PARSE_ADDRESS(CMD_ARGV[6], feapi_info) // feapi_info->base = 0x20000000
flash_bank_add() // 将 feapi_flash 加入 flash_banks 数组。
```

图 21: FESPI 解析

3.2 Flash Handler 函数注册

在 OpenOCD 中会有两类 handler 函数会被注册, 刚才讲的 config 类的 handler 函数部分, 到这里就该涉及到 flash 相关的执行类 handler 函数的注册了. 因为 flash_exec_command_handlers[] 实在太长, 只能截取一部分放在这里, 而且大家也注意到了图22是 flash_init_drivers() 函数, 它里面会对这个数组进行注册.

```
static int flash_init_drivers(struct command_context *cmd_ctx)
{
    if (!flash_bank_list())
        return ERROR_OK;

    return register_commands(cmd_ctx, "flash", flash_exec_command_handlers);
}
```

图 22: Handler 注册

那么 `flash_init_drivers()` 函数在哪里被调用呢? 看图23, `openocd_main()` 函数已经属于比较顶层的函数了, 属于比较早的启动代码了. 它做了两件事情: 一个就是调用 `setup_command_handler()` 函数, 主要完成对于 `config` 类 `handler` 的注册; 另一个就是调用 `openocd_thread()` 函数开始主线程循环, 同时进行执行 `handler` 函数的注册. 它会调用 `handle_init_command()` 函数, 在该函数中会有专门调用“flash init”字符串相关的 `handler` 函数.

[illegible]

图 23: Handler 顶层注册

关于 `handle_flash_init_command()` 怎么进行注册的一个关系, 当然它是在 `config` 类的 `handler` 函数注册时就完成的工作。`openocd_thread()` → `handle_init_command()` → `command_run_line("flash init")` 的调用关系使得 `flash_exec_command_handlers[]` 中的函数都得到了注册。

3.3 FLASH 刷写准备

在开始将 image 文件中的数据刷写到 flash 上之前,OpenOCD 自己需要完成一个重要的工作,那就是编译出一个可以运行在目标处理器上的小程序,这个小程序很简单,主要的逻辑就是操作处理器中关于 SPI 的一组寄存器,将来自 OpenOCD 的 image 数据通过 SPI 总线写入到 FLASH 中. 对于 RISC-V 平台来讲,目前它是放在这个路径下的:”riscv-openocd/contrib/loaders/flash/fespi/”,它最终会生成两个版本的 binary 文件: riscv32_fespi.inc 与 riscv64_fespi.inc 分别对应 32 位与 64 位的版本. 这两个 bin 文件可以在编译 OpenOCD 工具的时候自动生成,也可以在该目录下手动编译生成. 打开它里面的 Makefile 文件,很多的疑问迎刃而解: 首先它使用了”-fPIC”的编译选项,保证了生成出来的代码是与地址无关的,那么不管这个代码下载到哪个地址边界处,它都是可以正常运行的. 其次它使用了”-Obinary”的选项,将 ELF 文件不需要的垃圾信息都清除掉,只留下最小最干净的 binary. 最后它使

用了”-nostdlib” 以及”-nostartfiles”, 因此它自己写了一个很小的入口汇编文件”riscv_wrapper.S” 作为替代。

这样一个干净小巧的可运行在 RISC-V 平台上代码就完成了. 再多说几句关于这个运行在处理器上的小程序的处理逻辑. 打开它的文件”riscv_wrapper.S” 如下图所示, 因为代码很短我就直接全部贴出来并加以说明: 首先 `stack` 的空间它是已经开辟好了, 在程序的入口首先设置好了栈寄存器 `sp` 的值, 为以后进行函数调用做准备; 在下面截图的第 15 行进行了函数调用, 进入到函数 `flash_fespi()` 中, 这个函数是这个片上小程序唯一的主体函数, 刷写 FLASH 的工作就是在这里完成的. 注意它的入口参数有六个, 一定要记住, 后面我们会讲到. 在图24的第 16 行有个 `ebreak` 指令, 这个非常关键. 因为 `flash_fespi()` 函数不是个死循环, 在完成 `flash` 的刷写工作后, 它会从该函数中返回 (当然返回时, ”a0” 寄存器中带回了刷写过程中的状态). 为了让 OpenOCD 知道这个刷写的动作已经完成, 也为了防止片上程序乱飞, 此处 `ebreak` 指令的执行会将当前 Core 给 Halt 住, 同时切换当前模式为 Debug 模式, 并等待 OpenOCD 的垂询.

```
1  #if __riscv_xlen == 64
2  # define LREG ld
3  # define SREG sd
4  # define REGBYTES 8
5  #else
6  # define LREG lw
7  # define SREG sw
8  # define REGBYTES 4
9  #endif
10
11      .section .text.entry
12      .global _start
13  _start:
14      lla    sp, stack_end
15      jal    flash_fespi
16      ebreak
17
18      .section .data
19      .balign REGBYTES
20  stack:
21      .fill  16, REGBYTES, 0x8675309
22  stack_end:
23
```

图 24: Flashloader 启动文件

在 OpenOCD 启动之后, 在它的启动 Terminal 窗口, 会显示出 3333/4444/5555 三个端口, 我们需要使用的是 Telnet 端口, 也就是 4444 端口. 如果读者不喜欢这个数字, 其实是可以自行修改的 (如下所示), 修改完成后, 重新编译生成 OpenOCD 即可.”riscv-openocd/src/server/telnet_server.c” 文件中的 `telnet_register_commands()` 中: `telnet_port = strdup(”4444”);`

启动 Telnet 的命令步骤如下, 即可将 telnet 与 OpenOCD 建立起通信来, 他们之间的通信连接方式也是 socket 的方式, 同 GDB 与 OpenOCD 之间的方式一样的.

- 新开启一个 Terminal 窗口
- 在窗口中输入命令: ”telnet localhost 4444” 或者 ”telnet 127.0.0.1 4444”

3.4 Telnet 刷写 flash

3.4.1 PROBE 命令:

首先, 需要在 Telnet 的 Terminal 窗口中输入如下命令:

```
flash probe <bank-number>
```

此时如果 FLASH 设备上有对应的 bank 号, 那么就会打印出对应的 flash 设备的信息. 由该命令的前两个字符串"flash probe", OpenOCD 可以找到已注册的 handler 函数 handle_flash_probe_command(). 该函数的调用栈如图25所示. 通过 CMD_ARGV[0] 中存放的 bank-name 从全局变量 flash_banks[] 中查找对应的 flash_bank{} 实例. 然后从预定义数组 target_devices[] 中找到对应的项, 该项包含了重要的信息: 在 CPU-Core 上 SPI 寄存器空间的基地址, 后续需要访问 CPU 上的 SPI 总线寄存器都需要根据该基地址.

```
COMMAND_HANDLER handle_flash_probe_command
CALL_COMMAND_HANDLER(flash_command_get_bank_maybe_probe, 0, &p, false) "in tcl.c 文件中"
get_flash_bank_by_name_noprobe() "获取 flash-bank 从 flash_banks[] 全局变量中"
p->driver->probe()
fespi_probe()
    "由 target_devices[] 数组中找到对应的项, 确定Core上的SPI寄存器空间的基地址"
    "先确定好SPI寄存器的基地址"
    fespi_write_reg() "设置FESPI_REG_TCTRL寄存器为: FESPI_TMRM(1)"
    target_write_u32()
    target_write_memory() "操作Core上SPI的特定寄存器, 就像比写内存空间一样"
    fespi_set_dir() "设置FESPI_DIR_TCTRL寄存器, 即传输方向"
    fespi_disable_hw_mode()
    fespi_read_flash_id() "获取flash-id必须在关闭hardware模式的情况下"
    fespi_enable_hw_mode()
    "根据flash_id, 读取flash_id, 从flash_devices[]全局表中找到对应的项"
    "返回存储在flash的各个硬件的信息: sector_size, page_size, read_command/pprog_command/chip_erase_command等command命令的值"
```

图 25: Flash Probe 解析

要从 target_devices[] 数组中找到对应的表项, 主要是通过"tap idcode"来查找的, 这个 IDCODE 就是存放在 Debug-Module 寄存器中的一个 ID 寄存器 (在 RISC-V 平台上, 就是 IDCODE 0x01 寄存器), 存放在 target_devices[] 数组中的值要与该 IDCODE 寄存器中的值一致才可以被搜索到. 图26是该数组的内容, 它每一行属于一项内容, 每项又有三个 element, 我们刚才讲的是指第二个 element 的值要匹配. 那么第三个 element 就是我们说的"ctrl_base", 它就是 SPI 寄存器域的基地址, 这个信息需要查询相应的 CPU 的手册才能获得. 该表存放在 riscv-openocd/src/flash/nor/fespi.c 文件中.

```
137 static const struct fespi_target target_devices[] = {
138     /* name, tap_idcode, ctrl_base */
139     { "Freedom E310-G000 SPI Flash", 0x10e31913, 0x10014000 },
140     { "Freedom E310-G002 SPI Flash", 0x20000913, 0x10014000 },
141     { NULL, 0, 0 }
142 };
```

图 26: SPIFLASH 数据结构

在确定好了 SPI 的基地址寄存器后, 可以看到后续又通过 SPI 总线访问到 FLASH 设备上的信息"flash-id". 通过该 ID 信息与下图表 (全局预配置表"flash_devices[]") 中的 device_id 进行匹配. 该表的内容包含各种 FLASH 设备的信息, 该表存放在 riscv-openocd/src/flash/nor/spi.c 文件中, 如图27所示.

```
33 /* Shared table of known SPI flash devices for SPI-based flash drivers. Taken
34  * from device datasheets and linux SPI flash drivers. */
35 const struct flash_device flash_devices[] = {
36     /* name, read_cmd, qread_cmd, pprog_cmd, erase_cmd, chip_erase_cmd, device_id,
37      * page_size, sector_size, size_in_bytes */
38     FLASH_ID("st m25p05", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00102020, 0x80, 0x8000, 0x10000),
39     FLASH_ID("st m25p10", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00112020, 0x80, 0x8000, 0x20000),
40     FLASH_ID("st m25p20", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00122020, 0x100, 0x10000, 0x40000),
41     FLASH_ID("st m25p40", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00132020, 0x100, 0x10000, 0x80000),
42     FLASH_ID("st m25p80", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00142020, 0x100, 0x10000, 0x180000),
43     FLASH_ID("st m25p16", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00152020, 0x100, 0x10000, 0x280000),
44     FLASH_ID("st m25p32", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00162020, 0x100, 0x10000, 0x480000),
45     FLASH_ID("st m25p64", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00172020, 0x100, 0x10000, 0x880000),
46     FLASH_ID("st m25p128", 0x03, 0x00, 0x02, 0xd8, 0xc7, 0x00182020, 0x100, 0x40000, 0x1800000),
```

图 27: Flash 设备信息

3.4.2 WRITE-FLASH 命令

在完成第一步的 probe 后, 就可以开始进行写 flash 的操作了, 它的命令如下:

```
flash write_image erase <your-image-file> <address> bin
# or
flash write_bank <bank_id> <your-image-file> <offset>
```

我们就分析"flash write_bank"这条命令, 另一条命令多了 erase 的操作, 写的动作其实是一样的. 通过这条命令字符串, OpenOCD 可以很顺利地找到预注册的 handler 函数 handle_flash_write_bank_command(), 图28是该函数的调用栈截图,



- 首先根据传递进来的 `bank_id` 通过查找 `flash_banks[]` 数组找到匹配项。
- 获取 `offset` 的值, 该值是相对于 `bank` 的 `offset`, 未来作为一个目的地址使用的。
- 打开输入的 `image-file` 的句柄, 并读取到本地新分配的 `buffer` 中
- 然后调用 `flash_driver_write()` 函数, 开始将存放好的 `image-file` 中的数据写入到特定的 `flash-bank` 中。

- 校验 flash 要写入的 bank 中的每个 sector 是否被保护了, 若是则无法继续写则返回
- 确定好后面要下载的片上运行小程序及其大小: riscv32_bin/riscv64_bin, 就是预备知识里面讲到的程序
- 调用 target_alloc_working_area() 函数: 先在 OpenOCD 中开辟一个 buffer, 大小是要下载的 bin 的大小. 建立相应的管理结构体. 调用函数 target_read_memory() 将 Core 上由 target->working_area_phys 指定的地址, 由 target->working_area_size 指定大小的这块区域的数据读取到本地刚开辟的 buffer 中, 备份起来.
- 调用 target_write_buffer() 函数, 将小程序 bin(SPI 的驱动代码) 写入到 target->working_area_phys 指定的起始地址处. 如果写的过程中发生失败, 那么调用 target_free_working_area() 函数将备份的数据恢复到 Core 中去.
- 调用 target_get_working_area_avail() 函数中, 将最终要写入的 image-file 那么大的一块数据这样的空间对应的一个管理实例给开辟出来.
- 再次调用 target_alloc_working_area() 函数, 这次开辟的数据区是为了要下载 image-file 中的数据到 FLASH 中而分配的, 因为先由 OpenOCD 下载数据到 Core 的 working-area 中, 最后才有片上小程序将 working-area 中存放的 image-file 中的数据写入到 FLASH 中去的. 所以 Core 中的 working-area 上的这块区间的数据也需要备份到 OpenOCD 的本地 buffer 中.
- 在调用 target_write_buffer() 函数写入 image-file 中的数据之前, 首先需要配置片上小程序在运行时的那六个入参 (这个在预备知识中已经提及).
- 这里就开始调用 target_write_buffer() 将 image-file 中的数据写入到 Core 中由 configure 文件中指定的 working-area 中了.
- 调用 target_run_algorithm() 函数, 同时把预先配置好的含有六个入参的结构发送到 Core 中去, 写入到 Core 中的 a0~a5 这六个寄存器中, 作为片上小程序在 "jal flash_fespi" 时的入参. 这六个参数包含了 SPI 寄存器域的基地址, 要访问的 image-file 临时存放的物理地址以及它的大小.

- 当片上小程序完成将 image-file 的数据由 working-area 通过 SPI 总线写入到 FLASH 这个过程后, 就会从 flash_fespi() 函数返回后, 就会执行”ebreak”指令而 Halt 住.
- 其实 target_write_buffer() 以及 target_run_algorithm() 函数都是多次被调用的, OpenOCD 写入一部分数据到 Core 中的 DDR 中, 就会触发片上小程序将这一部分数据搬移写入到 FLASH 中, 这也是为了避免如果 image 文件过大, 尝试在一次搬移中就完成的话, 那么写入的时间就会过长, 那样可能会出现问題, 所以不得不切割成多次来处理.
- 在每次运行 target_run_algorithm() 函数时, OpenOCD 都会持续的 polling 当前 Debug-Module 的状态, 如果 polling 到的状态是”RISCV_HALT_BREAKPOINT”, 那么就表示当前的这次传输完毕了. 就可以调整参数进行下一次的传输了. (其实这部分在文章已经涉及到了)
- 完成所有的传输后, 分别调用两次 target_free_working_area() 对备份到 OpenOCD 本地的两个数据区恢复到 Core 上对应的区间中去. 等于片上小程序以及 image 的缓存区的数据就被覆盖掉了.

3.4.3 校验数据

完成上述的写入操作之后, 就是最后一步校验的工作, 它的命令如下:

```
flash verify_bank <bank_number> <your-image-file> <offset>
```

OpenOCD 通过命令字串”flash verify_bank”可以找到预注册的 handler 函数 handle_flash_verify_bank_command(). 它实际的操作很简单就是重新打开 image-file, 读出其中的数据, 将其与从 FLASH 中读取的数据进行比较即可.

3.5 小结

以上就是对 OpenOCD 所支持的 FLASH 烧写逻辑的分析. 可以看到它主要采用了下载一个片上小程序到 CPU 上去的方式, 由小程序来完成数据的搬移工作. OpenOCD 先将要烧写的 image 写入到 CPU 中由 cfg 文件指定的 working-area 中, 然后由小程序通过 SPI 总线写入到 FLASH 中, 所有烧写工作完成后, 擦除痕迹!